

AegiSec2: Plataforma Avanzada de Telemetría de Seguridad y Contención Activa

Documentación de Arquitectura Técnica

22 de junio de 2026

Resumen

AegiSec2 es una plataforma de ciberseguridad de próxima generación diseñada para proporcionar visibilidad profunda y respuestas automatizadas frente a amenazas en entornos Linux. El sistema se compone de un agente de monitoreo de ultrabajo nivel desarrollado en Rust, un backend reactivo en Java 21 (Spring WebFlux) y un Dashboard en React. Este documento detalla la arquitectura, heurísticas de detección, mecánicas de contención en el kernel y el flujo asíncrono de la plataforma completa.

Índice

1. Introducción	2
2. Arquitectura Global del Sistema	2
3. El Agente de Monitoreo (Rust)	2
3.1. Sistema Sensor (<code>sensor</code>)	2
3.2. Dynamic Risk Scoring (DRS) Engine (<code>drs.rs</code>)	2
3.3. Módulo Aislador / Contención Activa (<code>isolator.rs</code>)	3
3.4. Telemetría (<code>telemetry.rs</code>)	3
4. Backend Central (Java 21 / Spring Boot)	3
5. Dashboard Frontend (React)	4
6. Landing Page Promocional Web	4
7. Despliegue e Infraestructura	4
8. Conclusión	4

1. Introducción

AegiSec2 trasciende el concepto de monitoreo pasivo tradicional para convertirse en un sistema de **Defensa Activa**. Utilizando tecnologías modernas y fuertemente tipadas, la plataforma está diseñada para la ingesta masiva de eventos en tiempo real, evaluando el riesgo de entidades de manera dinámica y bloqueando intrusiones en milisegundos sin intervención humana.

2. Arquitectura Global del Sistema

El repositorio AegiSec2 opera bajo una arquitectura distribuida Cliente-Servidor:

- **Agente (/agent):** Daemon en Rust instalado en los nodos.
- **Backend (/server/backend):** Servidor API y WebSocket en Java 21.
- **Dashboard (/server/dashboard):** SPA en React para los administradores.
- **Landing Page (/landing):** Interfaz promocional web preparada para despliegue Cloud (AWS).

3. El Agente de Monitoreo (Rust)

Desarrollado en Rust, el agente prioriza la seguridad de la memoria y la velocidad extrema. Utiliza el runtime asíncrono `tokio` para procesar miles de eventos del sistema operativo sin bloquear los hilos.

3.1. Sistema Sensor (sensor)

El agente no hace "polling" ineficiente; intercepta eventos a nivel del kernel y del sistema de archivos:

- **procfs & nix:** Lectura de memoria, estado de procesos y syscalls mediante el sistema de archivos `proc` virtual.
- **inotify:** Subsistema de Linux para el monitoreo íntegro de modificaciones en archivos críticos (FIM).

3.2. Dynamic Risk Scoring (DRS) Engine (drs.rs)

El DRS es el motor heurístico del agente. Mantiene un `HashMap` protegido por un `Mutex` asíncrono que asocia Entidades (UIDs o IPs) con un *Risk Profile* (Perfil de Riesgo).

El motor implementa un modelo de ponderación conductual (Behavioral Weighting) basado en los siguientes vectores de análisis:

1. **[H-01 / H-05] Análisis de Linaje de Procesos (Web & Reverse Shells):** El motor audita la relación jerárquica de los procesos. Si detecta anomalías severas, como un intérprete web (`apache`, `nginx`, `php`) o de scripting (`python`, `perl`) instanciando procesos interactivos (`bash`, `sh`) junto a syscalls de red, el sistema eleva de inmediato el perfil de riesgo a nivel crítico, disparando protocolos de contención automatizada.

2. **[H-02] Análisis de Densidad Temporal (Enumeration Velocity):** En lugar de bloquear la ejecución aislada de binarios administrativos (`whoami`, `netstat`, `id`), el motor rastrea la frecuencia temporal. Si identifica la invocación de múltiples herramientas de reconocimiento bajo el mismo UID dentro de una ventana de tiempo reducida, clasifica el comportamiento como un patrón de escaneo hostil, elevando proporcionalmente la puntuación de riesgo.
3. **[H-00 / H-04] Firmas de Alta Fidelidad y Red:** Identificación determinista de artefactos ofensivos conocidos (`nc`, `msfvenom`, `socat`) e intentos de conexión a puertos C2 anómalos, aportando peso severo a la evaluación global de la entidad.

Decaimiento Matemático Asíncrono (Decay Loop): Para mitigar la saturación por falsos positivos, un *task* asíncrono en `tokio` aplica una función matemática de decaimiento cada 60 segundos. Esto representa el enfriamiento natural del contexto: si una anomalía aislada no escala o no presenta continuidad temporal, el perfil de riesgo de la entidad retorna gradualmente a su línea base operativa.

3.3. Módulo Aislador / Contención Activa (`isolator.rs`)

Cuando el DRS supera el umbral crítico, el `Isolator` bloquea el ataque:

- **Terminación Quirúrgica (`kill_tree`):** Explora el sistema de archivos `proc` (específicamente la ruta `/proc/<pid>/stat`) mediante un recorrido *breadth-first* para identificar a todos los descendientes del proceso atacante. Para prevenir la evasión (procesos huérfanos), envía una señal `SIGSTOP` para congelar el árbol entero, y luego envía `SIGKILL` desde las hojas hasta la raíz.
- **Bloqueo de Red (C2 Cutoff):** Interrumpe las conexiones Command & Control (C2) usando `nftables`. Si `nftables` no está disponible, realiza un *fallback* automático a `iptables`, estableciendo reglas de descarte de tráfico (DROP) para el UID comprometido o la IP de destino en la cadena `OUTPUT`.

3.4. Telemetría (`telemetry.rs`)

El agente garantiza la entrega de alertas bajo carga extrema:

- **Cola Prioritaria (Priority Queue):** Eventos de contención y bloqueos saltan los búferes y se transmiten vía un HTTP POST inmediato al servidor.
- **Cola Normal (Batch Queue):** Eventos de baja prioridad se almacenan en un lote que se vacía cada N segundos o cuando alcanza 50 elementos.
- **mTLS (Mutual TLS):** A través de `request` y `rustls`, los agentes y el servidor autentican bidireccionalmente sus certificados criptográficos para evitar la inyección de eventos por agentes no autorizados (Zero-Trust).

4. Backend Central (Java 21 / Spring Boot)

El backend coordina la lógica de negocio, almacenando la historia forense y enviando información a la interfaz web.

- **Núcleo Reactivo:** Basado en Project Reactor (`spring-boot-starter-webflux`), procesa las solicitudes I/O de los agentes sin bloquear hilos del SO, permitiendo escalar a miles de agentes simultáneamente.
- **WebSockets y STOMP:** Para lograr la ansiada “actualización sin recargas”, el servidor empuja (push) las alertas en el instante que llegan desde el agente al Dashboard vía WebSockets.
- **Seguridad y Persistencia:** Implementa Spring Security con validación de JSON Web Tokens (JWT) sin estado. Los perfiles de riesgo y la auditoría forense se guardan permanentemente en una base de datos relacional **PostgreSQL** vía Spring Data JPA.

5. Dashboard Frontend (React)

El Dashboard proporciona el “Single Pane of Glass” para los analistas de seguridad SOC.

- Construido en **React** con **TypeScript** utilizando **Vite** como empaquetador para una experiencia de desarrollo veloz.
- Interfaz renderizada con **TailwindCSS**.
- Suscrito en tiempo real a los tópicos de WebSocket, visualizando el riesgo de las máquinas y gráficos en vivo a medida que los ataques son repelidos por el motor DRS.

6. Landing Page Promocional Web

El repositorio también alberga el módulo `/landing`, un proyecto Vite independiente creado específicamente para marketing o despliegues Cloud (como AWS S3). Cuenta con diseño *Premium Dark Mode*, animaciones CSS sutiles y prácticas SEO integradas, informando sobre las capacidades del sistema a los clientes.

7. Despliegue e Infraestructura

Todo el entorno servidor está *dockerizado*. El archivo `docker-compose.yml` coordina la creación de redes aisladas para PostgreSQL, el contenedor del Backend de Spring Boot y el contenedor del Dashboard de React, permitiendo levantar un entorno corporativo completo con el comando `docker-compose up -d`.

8. Conclusión

La sinergia entre el bajo nivel de Rust en el espacio del kernel de Linux y la escalabilidad asíncrona de Java en el backend, convierten a AegiSec2 en una herramienta inmensamente poderosa y reactiva, preparada para auditar infraestructuras en tiempo real bajo los estándares de la industria moderna.